

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ ПО Лабораторной работе №4
по дисциплине Программирование микроконтроллеров

Использование таймера и аппаратных прерываний.

Студент,
группы 5130201/40003

_____ Четвергов И. С.

Руководитель,
Преподаватель

_____ Вербова Н. М.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

1. Тема

Использование таймера для формирования заданного временного интервала и аппаратного прерывания для перевода микроконтроллера в режим пониженного энергопотребления.

2. Цель работы

Ознакомиться с основными методами формирования заданных интервалов времени и перевода микроконтроллера в режим пониженного энергопотребления. Закрепить навыки работы с осциллографом и оценочной платой MCBSTM32F200 в качестве измерительного генератора.

3. Постановка задачи

Используя библиотеки Keil μ Vision5, разработать программу для микроконтроллера (МК) STM32F200, которая при помощи таймера формирует периодическое попеременное включение и выключение светодиодов PG6 и PG7 с заданными временными характеристиками. При нажатии на кнопку «WAKEUP» программа должна переводить МК в спящий режим, а при её отпускании — пробуждать МК.

4. Теоретические данные

Для формирования и измерения временных интервалов в МК применяется аппаратная подсистема — таймер. Таймер автоматически отсчитывает начало и конец заданного процесса или его отдельных этапов.

МК STM32F200 оснащён восемью периферийными программируемыми 16-разрядными таймерами TIM1–TIM8 с перезагружаемыми счётчиками и программируемыми предделителями. Они применяются для измерения длительности входных импульсов и генерации прямоугольных сигналов. С помощью предделителей и контроллера тактирования RCC длительность импульсов и период их следования можно регулировать от нескольких микросекунд до нескольких миллисекунд. Таймеры работают независимо друг от друга и не потребляют общих ресурсов, но при необходимости могут синхронизироваться.

Помимо периферийных таймеров, в ядре микроконтроллера имеется системный таймер SysTick, изначально предназначенный для формирования временной базы ОСРВ. Тем не менее его периодические прерывания допускается использовать и в других целях. В микроконтроллерах на ядре Cortex время реакции на прерывание строго фиксировано.

Для настройки SysTick в файле `core_cm3.h` предусмотрена функция `SysTick_Config(ticks)`, принимающая коэффициент деления тактовой частоты. По умолчанию тактовая частота ядра составляет 12 МГц.

При переходе в режим сна или глубокого сна тактирование процессора Cortex-M3 останавливается; в режиме глубокого сна дополнительно отключаются флэш-память и основной тактовый генератор. Управление этими режимами осуществляется через регистр `SCB_SCR`, а конфигурация режима сна задаётся с помощью маски `SCB_SCR_SLEEPONEXIT_Msk` из файла `core_cm3.h`.

5. Алгоритм программы

5.1. Инициализация периферии

- Включение тактирования портов GPIOG и GPIOA через регистр `RCC->AHB1ENR`.
- Включение тактирования контроллера конфигурации системы SYSCFG через регистр `RCC->APB2ENR`.
- Настройка выводов PG6, PG7 как цифровых выходов; PA0 — как цифрового входа.

5.2. Настройка внешних прерываний (EXTI)

- Разрешение прерываний по линии EXTI0 через регистр `EXTI->IMR`.
- Настройка срабатывания по нарастающему фронту (`EXTI->RTSR`).
- Привязка вывода PA0 к линии прерывания через `SYSCFG->EXTICR[0]`.

5.3. Настройка системного таймера SysTick

- Вызов `SystemCoreClockUpdate()` для получения актуальной тактовой частоты.
- Конфигурация SysTick с периодом прерывания 1 мс через `SysTick_Config`.

5.4. Режим пониженного энергопотребления

- Установка бита `SLEEPDEEP` в `SCB->SCR` для выбора режима глубокого сна.
- Настройка регистров `PWR->CR` для выбора режима standby.

5.5. Управление прерываниями (NVIC)

- Задание групп и приоритетов прерываний от EXTI0 и SysTick.
- Активация обработчиков прерываний.

5.6. Основной цикл

В бесконечном цикле поочерёдно вызываются функции управления светодиодами: включается PG6, затем PG7, после чего цикл повторяется.

5.7. Обработчик прерывания EXTI0

При нажатии кнопки «WAKEUP» выполняется программная задержка для устранения дребезга, затем инвертируется бит SLEEPONEXIT — МК переходит в спящий режим или пробуждается. Фронт срабатывания прерывания переключается на противоположный.

6. Код программы

```
1 #include "stm32f207xx.h"
2 #include "stdbool.h"
3 #include "stdio.h"
4
5 // Программная задержка на основе пустого цикла
6 void delay(unsigned long cycles)
7 {
8     unsigned long i = 0;
9     for (i = 0; i < cycles; i++)
10     { }
11 }
12
13 // Счётчик миллисекунд, обновляемый системным таймером
14 volatile uint32_t msTicks = 0;
15
16 // Задержка в миллисекундах на основе аппаратного таймера
17 void delay_ms(uint32_t ms)
18 {
19     uint32_t t = msTicks + ms;
20     while (t > msTicks);
21 }
22
23 // Обработчик прерывания системного таймера SysTick — увеличивает
    счётчик на 1
24 void SysTick_Handler()
25 {
26     msTicks++;
27 }
28
29 // Обработчик внешнего прерывания по линии EXTI0 кнопка ( WAKEUP)
30 void EXTI0_IRQHandler(void)
31 {
32     // Программная задержка для подавлениядребезга контактов
33     delay(5000);
34
35     // Переключение бита SLEEPONEXIT: переход в сон или
    пробуждение МК
36     SCB->SCR ^= SCB_SCR_SLEEPONEXIT_Msk;
37
38     // Инверсия фронта срабатывания прерывания
39     EXTI->RTSR ^= EXTI_RTSR_TR0;
40     EXTI->FTSR ^= EXTI_FTSR_TR0;
41
42     // Сброс флага ожидающего прерывания
43     EXTI->PR |= EXTI_PR_PR0;
44 }
45
46 // Попеременное включение светодиодов PG6 и PG7
47 void DiodPG6(void)
48 {
```

```

49 // Включить светодиод на PG6
50 GPIOG->ODR |= 1ul << 6;
51 delay(500000);
52 // Выключить светодиод на PG6
53 GPIOG->ODR &= ~1ul << 6;
54
55 // Включить светодиод на PG7
56 GPIOG->ODR |= 1ul << 7;
57 delay(500000);
58 // Выключить светодиод на PG7
59 GPIOG->ODR &= ~1ul << 7;
60 }
61
62 // Заготовка для дополнительного управления светодиодом PG7
63 void DiodPG7(void)
64 {
65
66 }
67
68 int main()
69 {
70 // Включить тактирование порта GPIOG
71 RCC->AHB1ENR |= 1ul << 6;
72 // Включить тактирование порта GPIOA
73 RCC->AHB1ENR |= 1ul << 0;
74 // Включить тактирование контроллера конфигурации системы
75 SYSCFG
76 RCC->APB2ENR |= 1ul << 14;
77
78 // Настроить PG6 как цифровой выход
79 GPIOG->MODER = (GPIOG->MODER & ~(1ul << 13)) | 1ul << 12;
80 // Настроить PG7 как цифровой выход
81 GPIOG->MODER = (GPIOG->MODER & ~(1ul << 15)) | 1ul << 14;
82 // Настроить PG8 как цифровой выход
83 GPIOG->MODER = (GPIOG->MODER & ~(1ul << 17)) | 1ul << 16;
84 // Настроить PA0 как цифровой вход кнопка ( WAKEUP)
85 GPIOG->MODER &= ~GPIO_MODER_MODER0;
86
87 // Разрешить прерывания по внешней линии EXTI0
88 EXTI->IMR |= EXTI_IMR_MR0_Msk;
89 // Настроить срабатывание по нарастающему фронту нажатие (
90 кнопки)
91 EXTI->RTSR |= EXTI_RTISR_TR0;
92 // Отключить срабатывание по спадающему фронту
93 EXTI->FTSR &= ~EXTI_RTISR_TR0;
94
95 // Привязать вывод PA0 к линии прерывания EXTI0
96 SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI0_PA;
97
98 // Обновить значение системной тактовой частоты
99 SystemCoreClockUpdate();
100 // Настроить SysTick с периодом прерывания 1 мс
101 if (SysTick_Config(SystemCoreClock / 1000))

```

```

100 {
101 }
102
103 // Активировать режим глубокого сна через бит SLEEPDEEP
104 SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
105 // Настроить регистр питания: сбросить бит PDDS
106 PWR->CR &= PWR_CR_PDDS;
107 // Выбрать режим пониженного потребления standby
108 PWR->CR |= PWR_CR_LPDS;
109
110 // Задать схему группировки приоритетов прерываний
111 NVIC_SetPriorityGrouping(5);
112 // Установить наивысший приоритет для EXTI0
113 NVIC_SetPriority(EXTI0_IRQn, 0);
114 // Установить приоритет для SysTick ниже (, чем у EXTI0)
115 NVIC_SetPriority(SysTick_IRQn, 4);
116
117 // Активировать прерывание от SysTick
118 NVIC_EnableIRQ(SysTick_IRQn);
119 // Активировать прерывание от внешней линии EXTI0
120 NVIC_EnableIRQ(EXTI0_IRQn);
121
122 // Основной цикл: циклическое мигание светодиодов
123 while (1)
124 {
125     DiodPG6();
126     DiodPG7();
127 }
128 }

```

Listing 1: MyBlinky.c

7. Работа с осциллографом

Программное измерение периода, частоты сигнала, ширины положительного и отрицательного уровней.

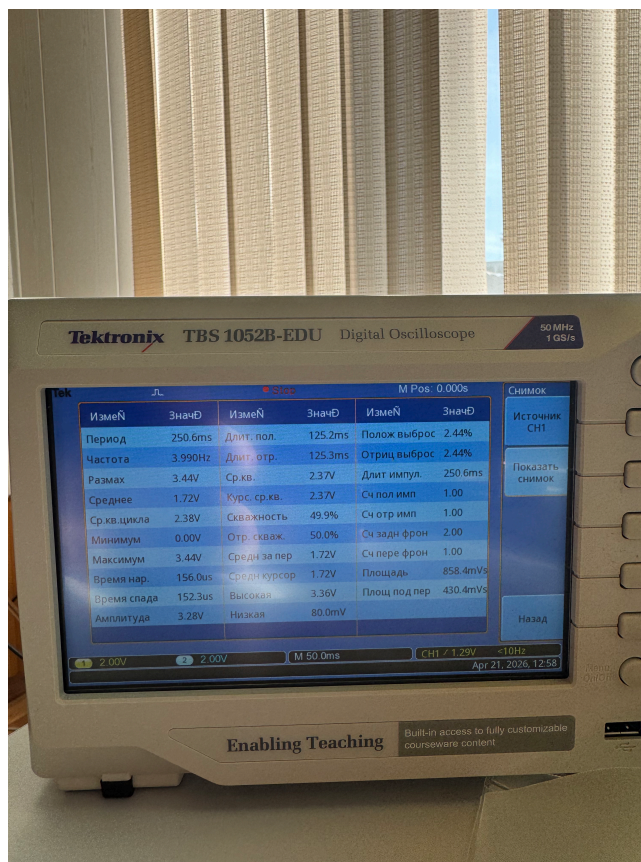


Рис. 1: Программное измерение характеристик сигнала

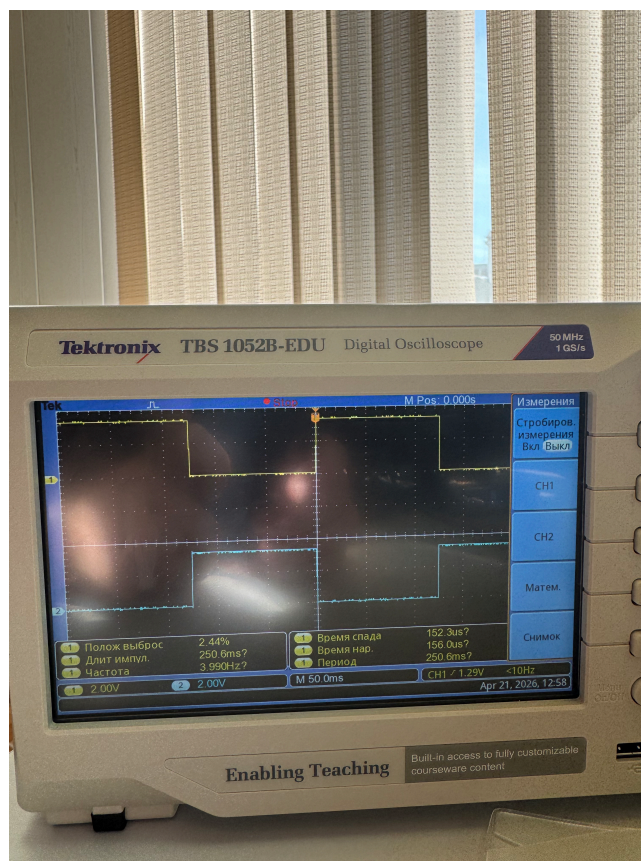


Рис. 2: Осциллограммы двух каналов с измеренными параметрами

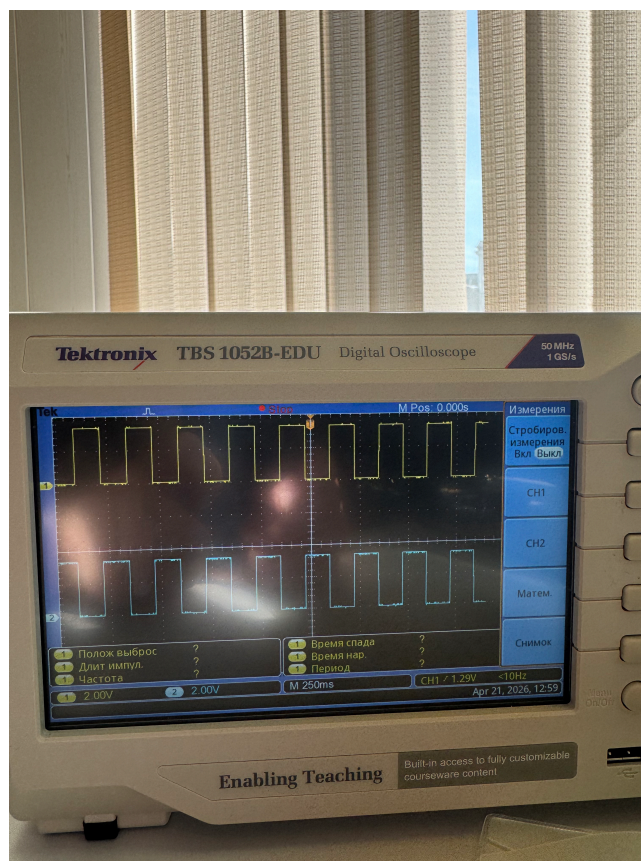


Рис. 3: Попеременное переключение сигналов на двух каналах

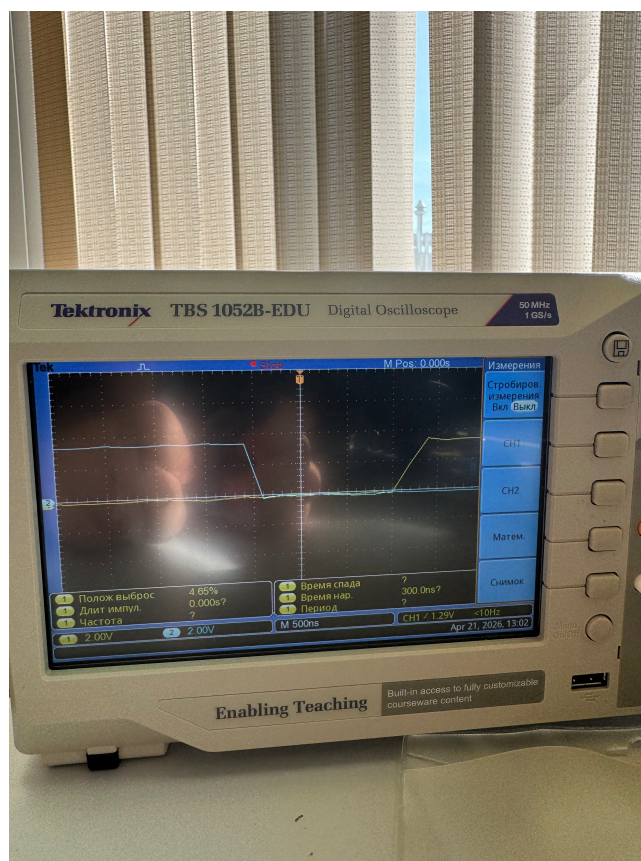


Рис. 4: Спад первого сигнала и нарастание второго

8. Результаты работы с осциллографом

По результатам измерений на осциллографе Tektronix TBS 1052B-EDU получены следующие характеристики сигнала:

- Период: 250.6 мс
- Частота: 3.990 Гц
- Размах: 3.44 В
- Амплитуда: 3.28 В
- Длительность положительного уровня: 125.2 мс
- Длительность отрицательного уровня: 125.3 мс
- Скважность: 49.9%
- Время нарастания: 156.0 мкс
- Время спада: 152.3 мкс

Заключение

В ходе выполнения лабораторной работы освоены основные методы формирования временных интервалов с использованием системного таймера SysTick, а также методы перевода микроконтроллера в режим пониженного энергопотребления. Разработана программа на языке С в среде Keil μ Vision5, обеспечивающая попеременное включение светодиодов PG6 и PG7 с заданными характеристиками, а также обработку нажатия кнопки «WAKEUP» для перехода МК в спящий режим и его пробуждения. Закреплены навыки работы с осциллографом и оценочной платой MCBSTM32F200 в качестве измерительного генератора.